

Machine Learning in Practice

Saket Anand

Outline

- Machine Learning in practice
 - Performance Evaluation
 - Diagnostics for Learning Algorithms
 - Analysis of Learning Systems

Performance Evaluation of Learning Tasks

- Measuring Performance: How well does a learned model work?
 - Performance is typically measured by estimating the TRUE ERROR RATE, the classifier's error rate on the ENTIRE POPULATION.
- Evaluation Metrics
 - Classification: Accuracy
 - Regression: Mean Squared Error
 - Retrieval: Precision/Recall, F-Score
 - Ranking: mean Average Precision
 - Clustering: NMI

Regression Metrics

- Mean Squared Error

$$\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$$

- Mean Absolute Error

$$\frac{1}{N} \sum_{i=1}^N |\hat{y}^{(i)} - y^{(i)}|$$

- Others like Root Mean Squared Error, Max Error (L_∞ -norm), etc.

Definitions in the Classification context

- True Positive (TP)
- False Positive (FP)
 - Also called 'False Alarms'
- True Negative (TN)
- False Negative (FN)
 - Also called 'misses' as in hits & misses



Confusion Matrix

Actual Class		Predicted Class	
		Positive	Negative
		TP	FN
Actual Class	Positive	TP	FN
	Negative	FP	TN

Binary Classification Confusion Matrix

		Predicted			
		$C_0 \dots C_{k-1}$	C_k	$C_{k+1} \dots C_N$	
Actual	C_0	TN	FP	TN	True Positive False Negative True Negative False Positive
	...				
	C_{k-1}				
	C_k	FN	TP	FN	
	C_{k+1}				
...					
	C_N	TN	FP	TN	

Multi-Class Classification Confusion Matrix

Evaluation Metrics

- When classification is involved in some form,
 - the quantitative evaluation metric typically is a function of TP, TN, FP & FN
- Depending on the application (e.g., object detection / semantic segmentation / instance segmentation)
 - the method of computing TP, TN, FP & FN may change
- Classification is relatively simple
 - Accuracy: $(TP + TN) / (TP + FP + TN + FN) = (TP + TN) / (\text{\#samples})$
 - Precision (P): $TP / (TP + FP)$
 - Recall (R): $TP / (TP + FN)$
 - F1 score: $2P \times R / (P + R)$

Classification Metrics – Imbalanced Data

Not all Evaluation Metrics are the same;
Choice often depends on the application & data

- Class 0: Healthy Individuals
 - #samples: 9900
- Class 1: Individuals with Cancer
 - #samples: 100
- **Trivial Classifier:** All samples in Class 0
 - Accuracy: $(TP + TN) / (TP + FP + TN + FN) \sim 99\%$
 - Precision (P): $TP / (TP + FP) = 0 / (0 + 0) = \text{NaN} / \text{undefined}$.
 - Recall (R): $TP / (TP + FN) = 0 / (0 + 100) = 0\%$
 - F1-score $(2PR / (P + R))$: $0 / 0 = \text{NaN} / \text{undefined}$.
- **Bad classifier:** Most samples in Class 0
 - Accuracy: $(TP + TN) / (TP + FP + TN + FN) \sim 99\%$
 - Precision: $TP / (TP + FP) = 1 / (1 + 1) = 50\%$
 - Recall: $TP / (TP + FN) = 1 / (1 + 99) = 1\%$
 - F1-score $(2PR / (P + R))$: ~ 0.02
- **Perfect Classifier:**
 - Accuracy: $(TP + TN) / (TP + FP + TN + FN) = (100 + 9900) / (100 + 9900 + 0 + 0) = 100\%$
 - Precision: $TP / (TP + FP) = 100 / (100 + 0) = 100\%$
 - Recall: $TP / (TP + FN) = 100 / (100 + 0) = 100\%$
 - F1-score $(2PR / (P + R))$: 1

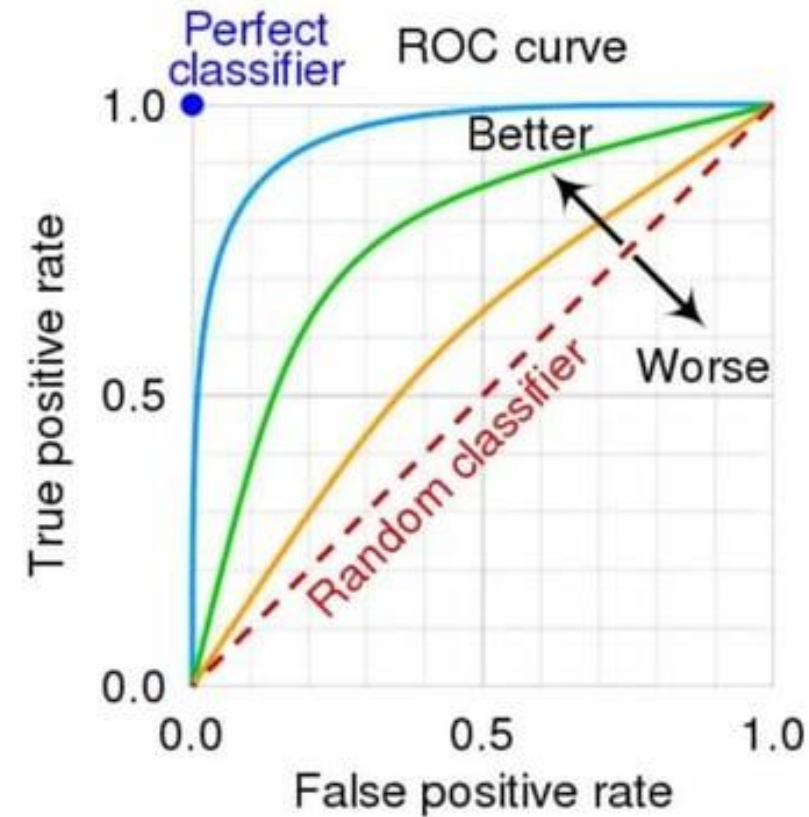
Perfect Classifier		Actual	
		Class 1	Class 0
Predicted	Class 1	TP 100	FP 0
	Class 0	FN 0	TN 9900

Trivial Classifier		Actual	
		Class 1	Class 0
Predicted	Class 1	TP 0	FP 0
	Class 0	FN 100	TN 9899

Bad Classifier		Actual	
		Class 1	Class 0
Predicted	Class 1	TP 1	FP 1
	Class 0	FN 99	TN 9899

Receiver Operating Characteristic (ROC*) Curve

- True Positive Rate (TPR) = Recall = $TP/(TP+FN)$
- False Positive Rate (FPR) = $FP / (FP + TN)$
- To plot the ROC curve for a binary classifier
 - Say the classifier generates $P(Y=1|X)$
 - Set a threshold $\tau \in [0,1]$ on $P(Y=1|X)$ over its range (e.g., $\{0, 0.01, 0.02, \dots, 1\}$)
 - Compute TPR & FPR for each τ
 - Plot the ROC curve.
 - Compute the Area under the curve (AUC-ROC)
- A random binary classifier has AUC-ROC = 0.5
- A perfect one has AUC-ROC = 1
- Similarly other plots – Precision vs. Recall curve, etc.



*The name comes from early days where detecting an object in a RADAR receiver was the primary task

Understanding Model Training and Evaluation

Performance Evaluation of Learning Tasks

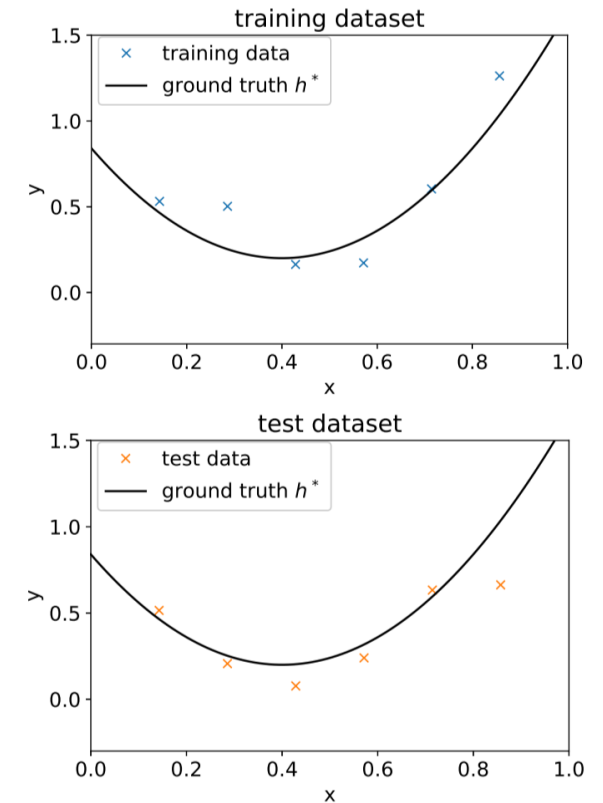
- Entire population is unavailable for performance evaluation
- Finite set of training data, usually smaller than desired
- Naïve approach: use all available data to train
 - The final model will typically **overfit** the training data
 - More pronounced with high-capacity models (e.g., neural nets)
 - The true error rate is **underestimated**
 - Not uncommon to have 100% accuracy on training data

Data Terminology

- Training Data (Train set):
 - The Data subset using which you train your model, i.e., the data samples over which you ‘maximize the likelihood’ / ‘minimize the loss’ to find the best model parameters
- Development / Validation Data (Dev / Val set):
 - The Data subset over which “hyperparameters” are tuned, comparisons are done, but models are not trained
- Test Data (Test set):
 - The “Unseen” Data subset over which the model performance shall be evaluated

Assumption: All these data subsets are drawn from the same data distribution!

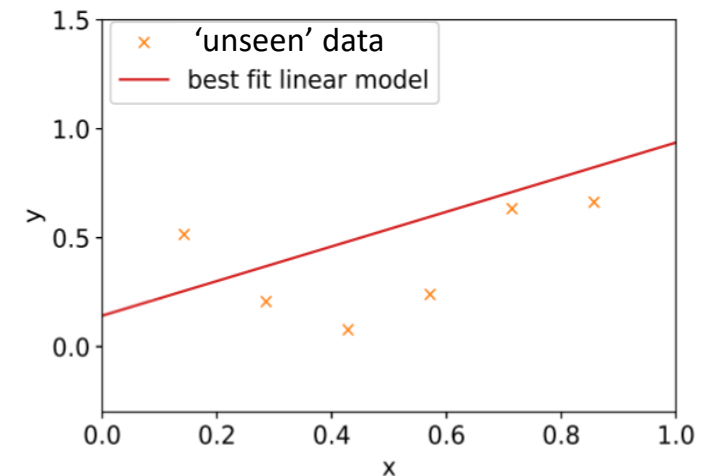
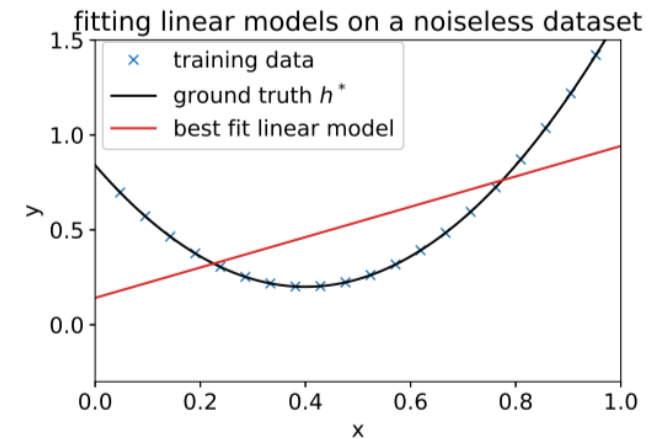
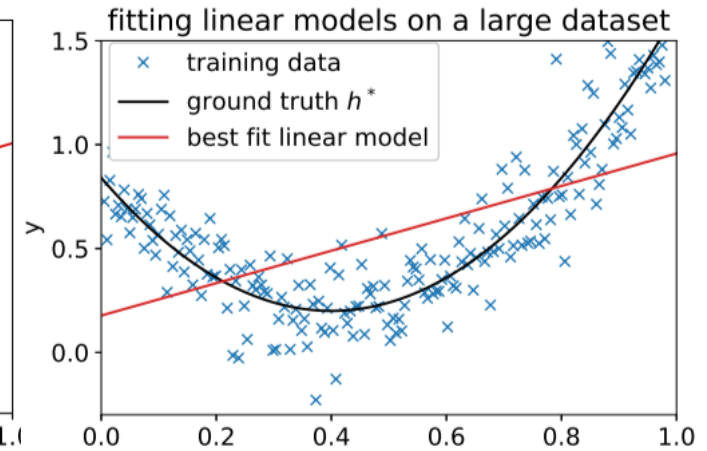
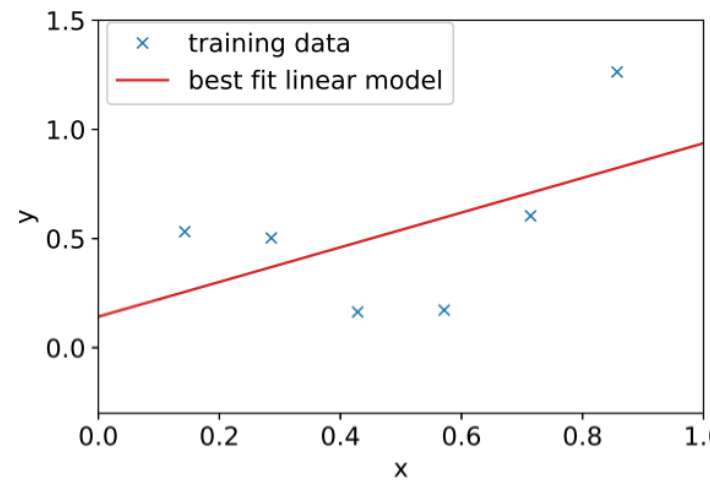
Data Generation Function:
Quadratic curve



Model Fitting

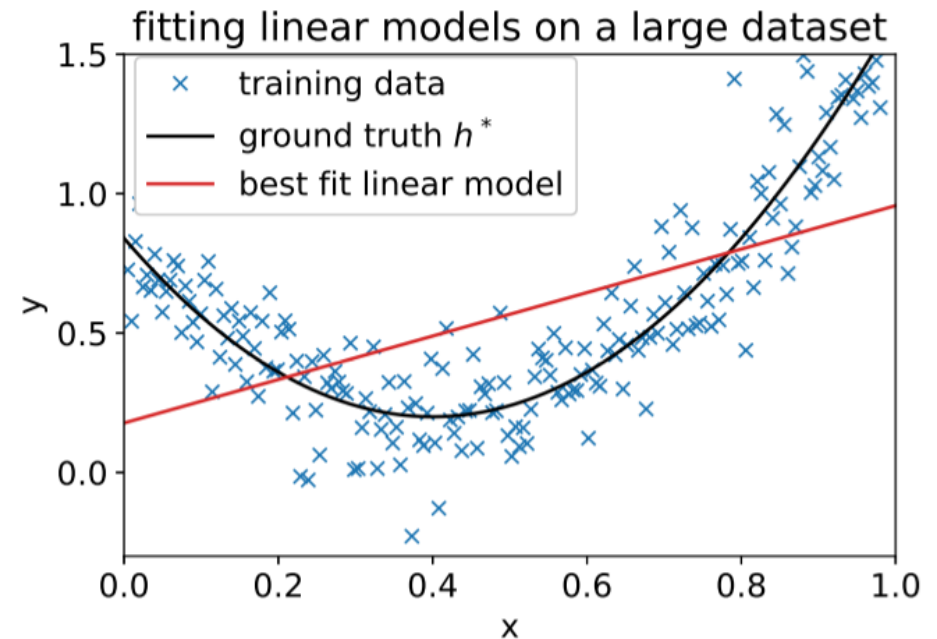
- First order curve: $y = w_0 + w_1 x$
- Observations
 - High error on Training set
 - High error on Val set
- What are the sources of error?
 - Small no. of training samples?
 - Noise in training data?
 - Insufficient Model capacity?
 - Too few parameters to capture data complexity
 - e.g., line used to model points along a quadratic curve

This is the case of “**underfitting**” where the model is not capable of sufficiently expressing the training data (i.e., high training error)

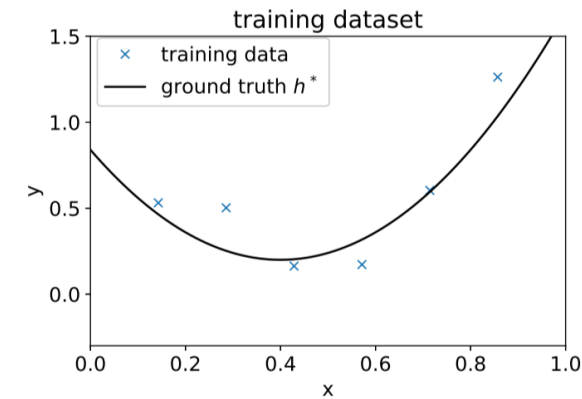


Bias and Underfitting

- Bias of a model:
 - Even with a very large training set, the val / test error is high
 - Characterized by a high training error and high val / test error
 - Indicates the model is underfitting the data



Model Fitting



- Fifth order curve: $y = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$
- Digression
 - Is the above a linear model?
 - Short answer: Yes, it is a linear model
 - Longer answer:
 - The question should be is the function $y=f(x,\mathbf{w})$ linear in $\mathbf{w}$?
 - Yes, it is. Since all the w_i terms have power one, this function is linear in $\mathbf{w}$.

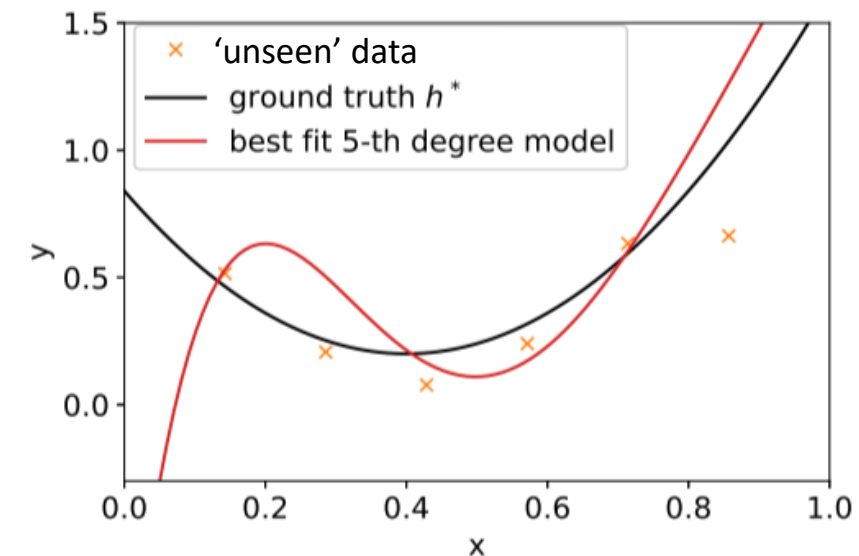
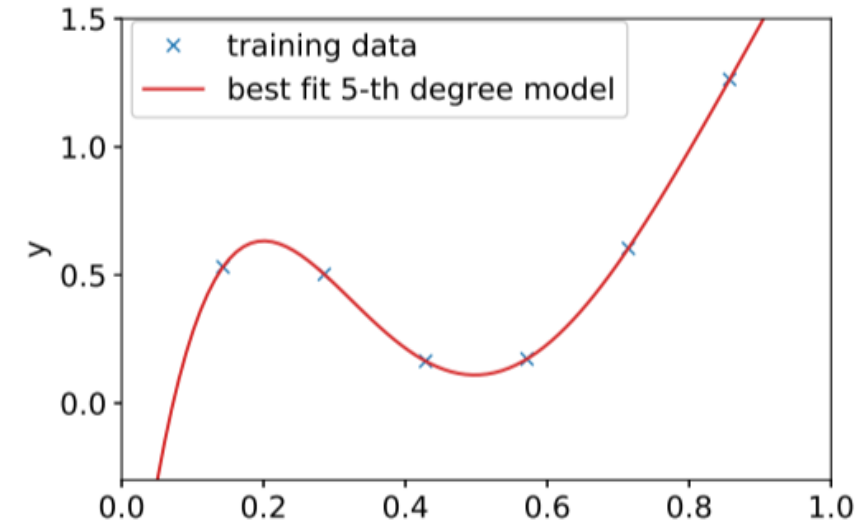
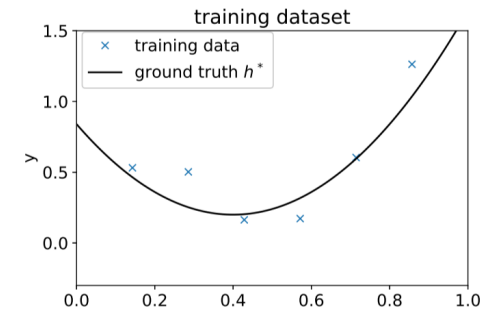
Linearity

- A function $g(u)$ is linear if it satisfies both of the following properties:
 - Superposition: $g(u+v) = g(u) + g(v)$
 - Homogeneity: $g(cu) = c g(u)$
- Our linear model of $y=f(\mathbf{x},\mathbf{w}) = \mathbf{w}^T\mathbf{x}$ is linear in $\mathbf{x}$ and in $\mathbf{w}$
 - Let's verify:
 - $f(\mathbf{x}_1 + \mathbf{x}_2, \mathbf{w}) = \mathbf{w}^T(\mathbf{x}_1 + \mathbf{x}_2) = \mathbf{w}^T\mathbf{x}_1 + \mathbf{w}^T\mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{w}) + f(\mathbf{x}_2, \mathbf{w})$
 - $f(c\mathbf{x}, \mathbf{w}) = \mathbf{w}^T(c\mathbf{x}) = c \mathbf{w}^T\mathbf{x} = c f(\mathbf{x}, \mathbf{w})$
 - The same arguments apply to $\mathbf{w}$
- What about $y=f_1(x, w_0, w_1) = w_0 + w_1x$?
 - Is $f_1(x, w_0, w_1)$ linear in x ?
 - Is $f_1(x, w_0, w_1)$ linear in $\mathbf{w}=[w_0, w_1]^T$?
- What about $y = f_2(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$?
 - Is $f_2(x, \mathbf{w})$ linear in x ? Is it linear in $\mathbf{w}$?

Model Fitting

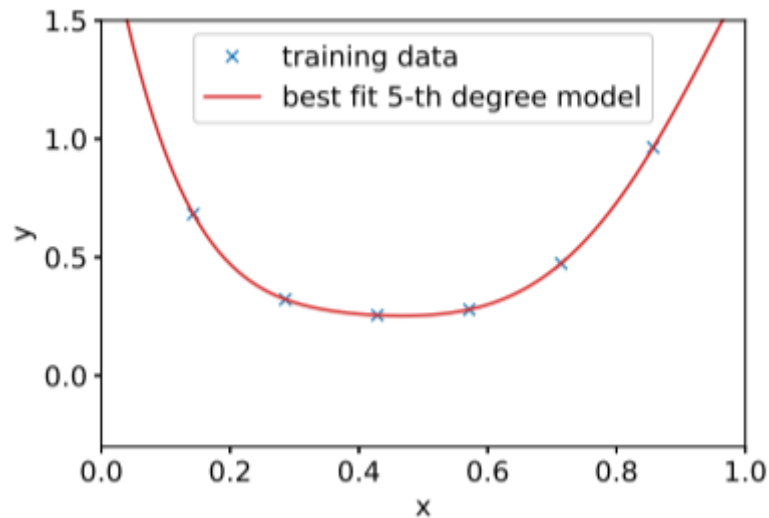
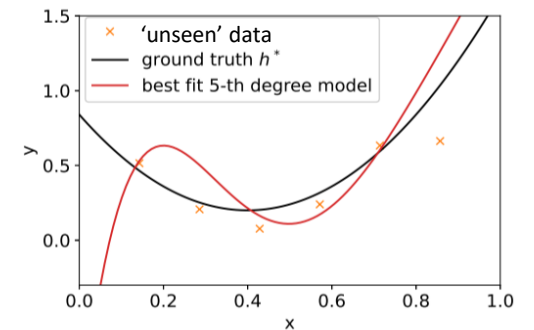
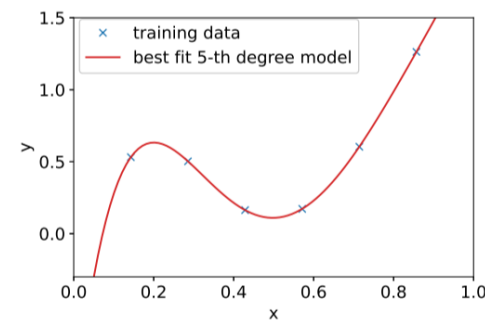
- Fifth order curve:
 - $y = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$
- Model fits perfectly to training data
 - Zero training error
 - Somewhat small-ish val / test error
 - Still very different from the ground-truth quadratic curve

We got lucky!

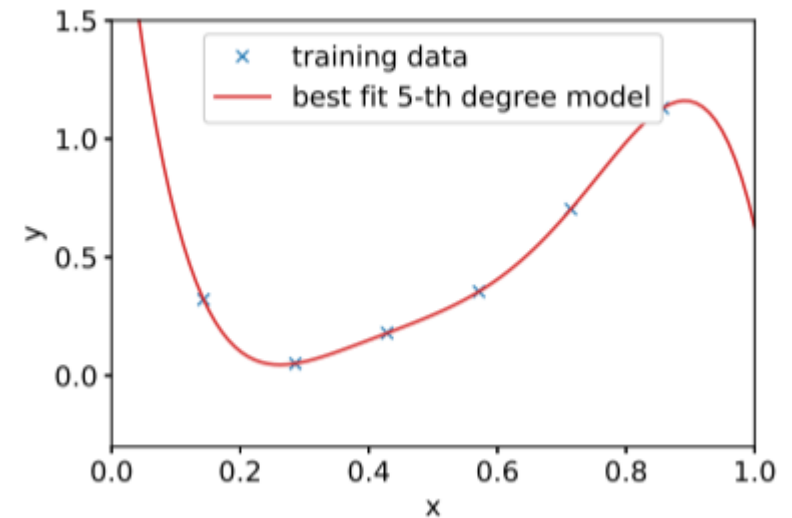
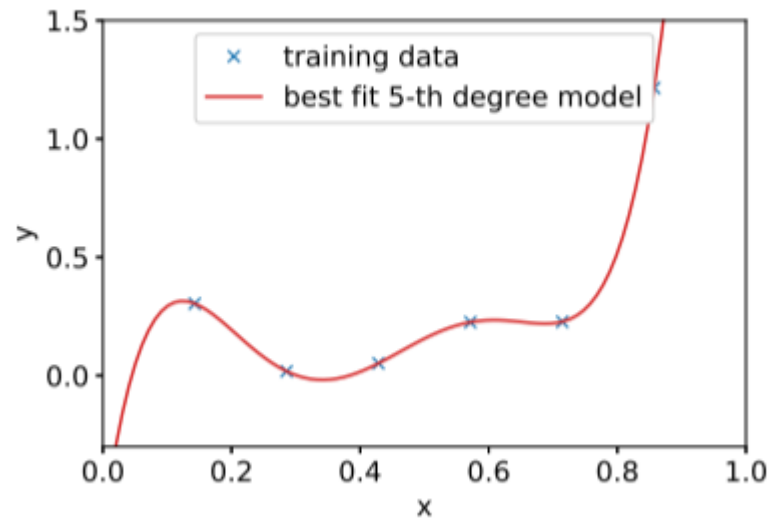


Model Fitting

- Fifth order curve:
 - $y = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$
- The model “overfits” the training data
 - i.e., the model fits the noise in that specific train set



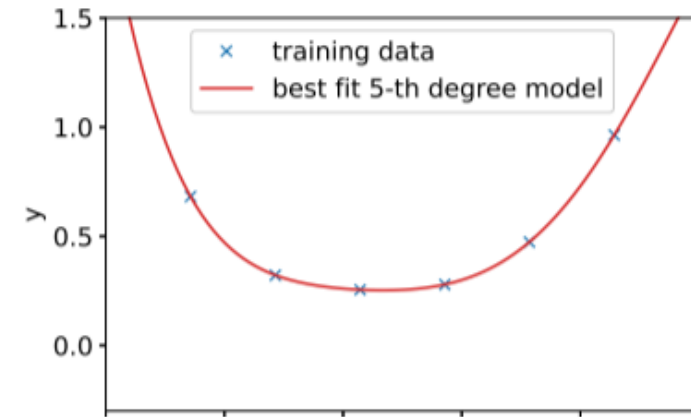
fitting 5-th degree model on different datasets



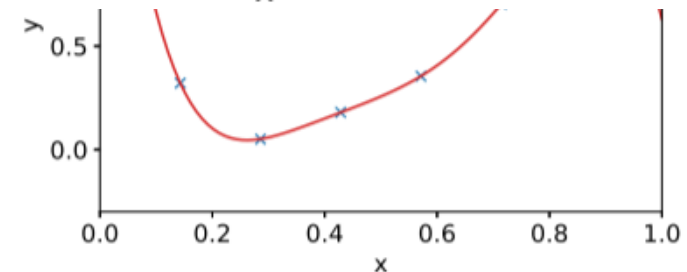
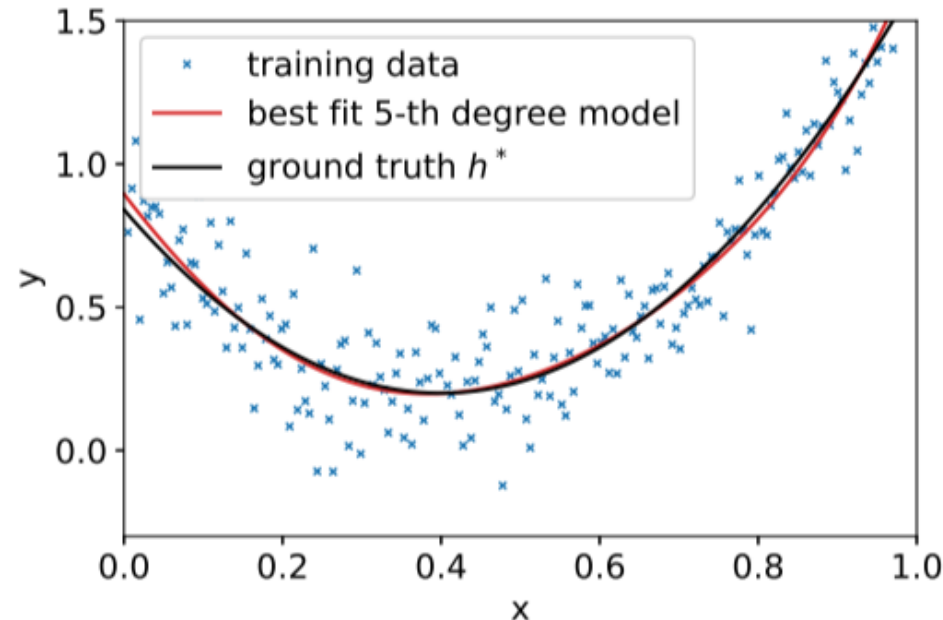
Variance and Overfitting

- Variance of a Model

- Fits the spurious patterns in the (usually small, finite) train set
 - Characterized by low train error, but high val / test error
- Indicates model is overfitting the train set
 - Typical of high capacity models, i.e., models with many parameters
- Several mitigation approaches exist
 - Use a larger train set
 - Regularization
 - Feature Engineering

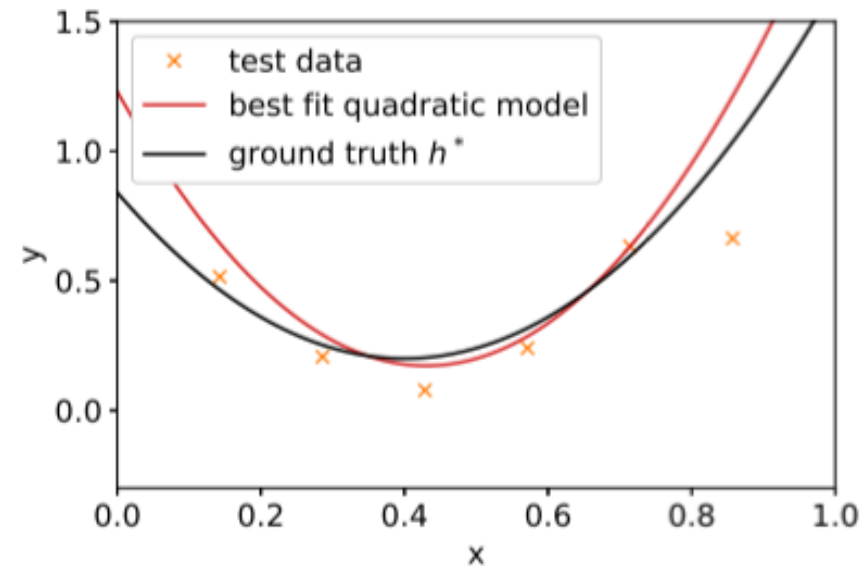
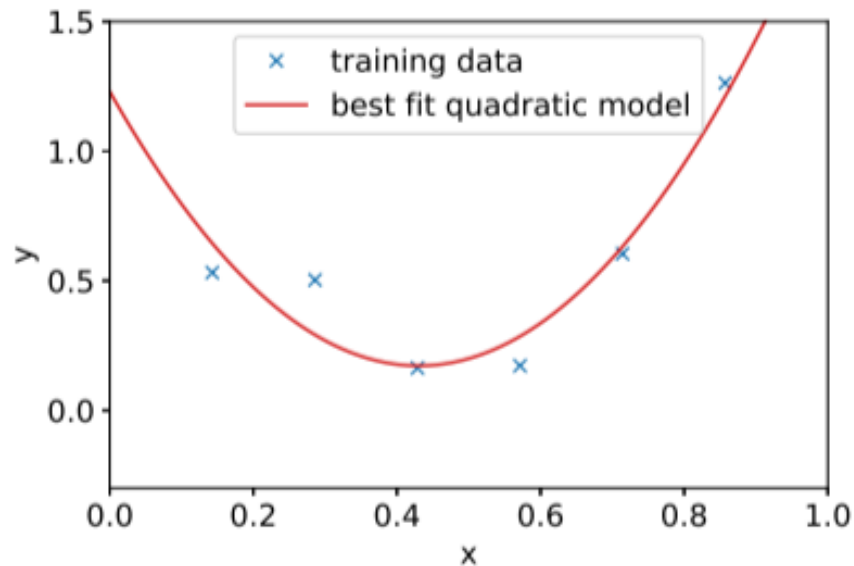


fitting 5-th degree model on large dataset

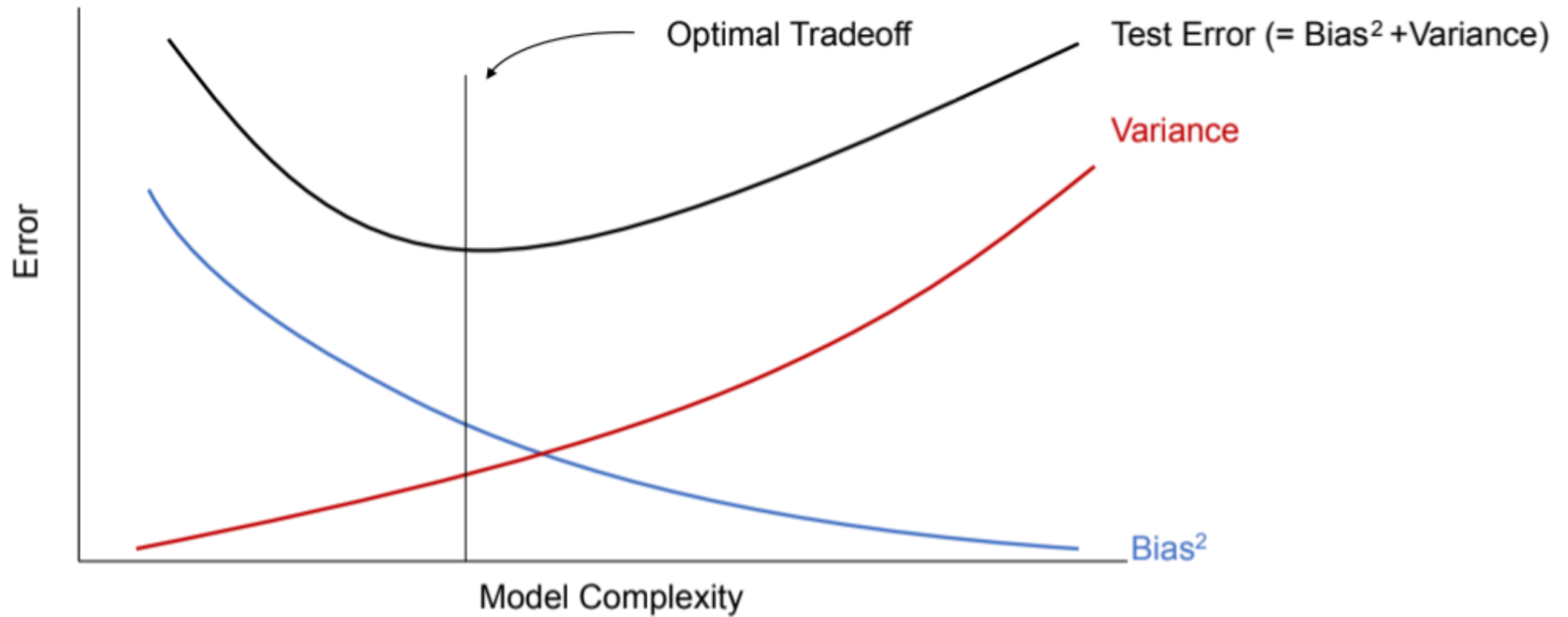


The Bias-Variance Trade-off

- High Bias / Underfitting: Model complexity is too low
 - Training error and Test / Val error are both too high
- High Variance / Overfitting: Model complexity is too high
 - Training error is low, but Test / Val error is too high
- Correct Fit: Model Complexity is just right
 - Both Training and Test / Val errors are low enough
 - In our running example, this is achieved by fitting a quadratic curve



The Bias-Variance Trade-off



How does Regularization help?

- Fifth order curve:

- $y = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$

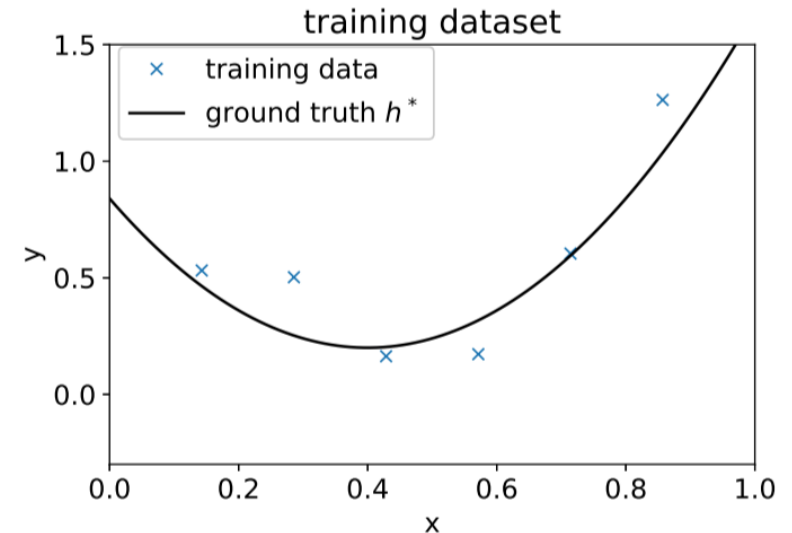
- What if $w_3=w_4=w_5=0$?

- How do we achieve it?

Let $\vec{w} = [w_0 \ w_1 \ w_2 \ w_3 \ w_4 \ w_5]^T$
& $\vec{x} = [1 \ x \ x^2 \ x^3 \ x^4 \ x^5]$
then the learned model has parameters

$$\vec{w}^* = \arg \min_{\vec{w}} \sum_{i=1}^N \left(y^{(i)} - \vec{w}^T \vec{x}^{(i)} \right)^2$$

s.t. $\|\vec{w}\|_0 \leq 3$



constraints on
the model parameters

How does Regularization help?

- Fifth order curve:

- $y = w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5$

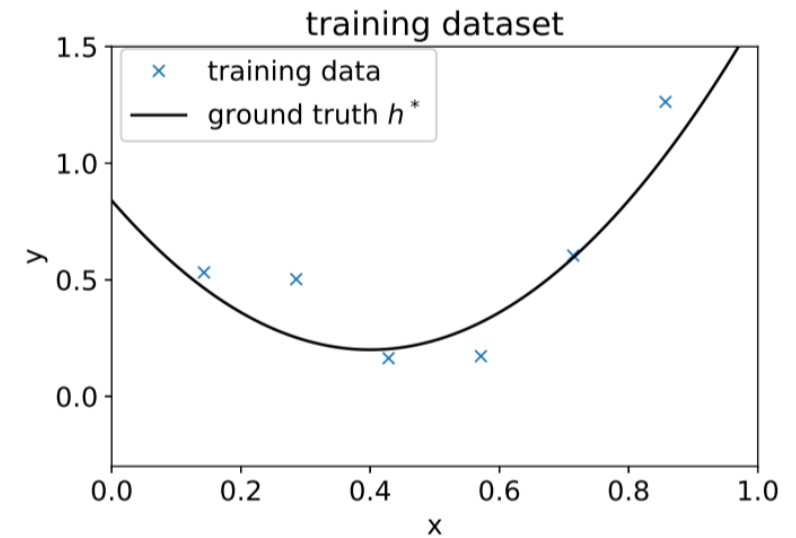
- What if $w_3=w_4=w_5=0$?

Let $\vec{w} = [w_0 \ w_1 \ w_2 \ w_3 \ w_4 \ w_5]^T$
 & $\vec{x} = [1 \ x \ x^2 \ x^3 \ x^4 \ x^5]$
 then the learned model has parameters

$$\vec{w}^* = \arg \min_{\vec{w}} \sum_{i=1}^N (y^{(i)} - \vec{w}^T \vec{x}^{(i)})^2 + \underbrace{\lambda \|\vec{w}\|_0}_{\text{Lagrange multiplier}}$$

∴ Relaxed optimization problem is used

$$\vec{w}^* = \arg \min_{\vec{w}} \sum_{i=1}^N (y^{(i)} - \vec{w}^T \vec{x}^{(i)})^2 + \underbrace{\lambda \|\vec{w}\|_1}_{\text{Sparsity promoting regularization term (also differentiable \& convex)}}$$



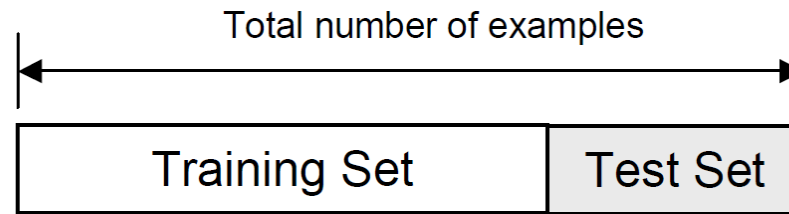
Performance Validation

How to ensure our model is trained *properly*?

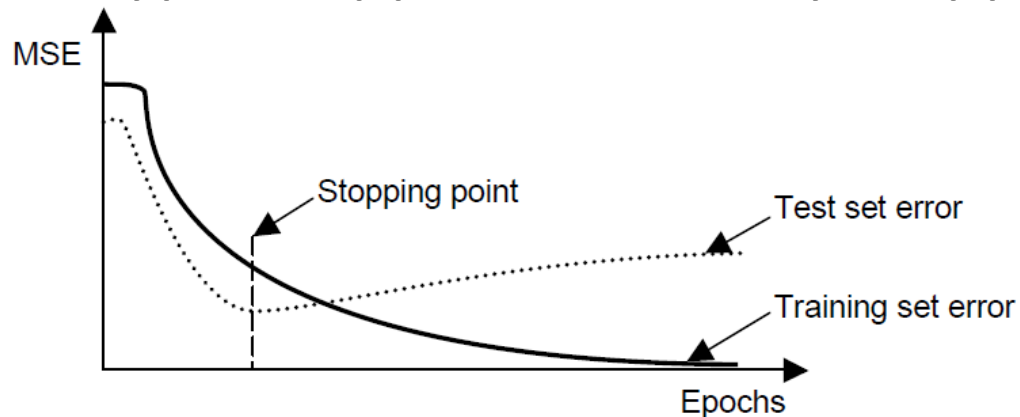
i.e., it is neither underfitting nor overfitting

Validation Method: Holdout

- Split dataset into two groups
 - Training set: used to train the model
 - Test set: used to estimate the error rate of the trained model



- Typical application: early stopping

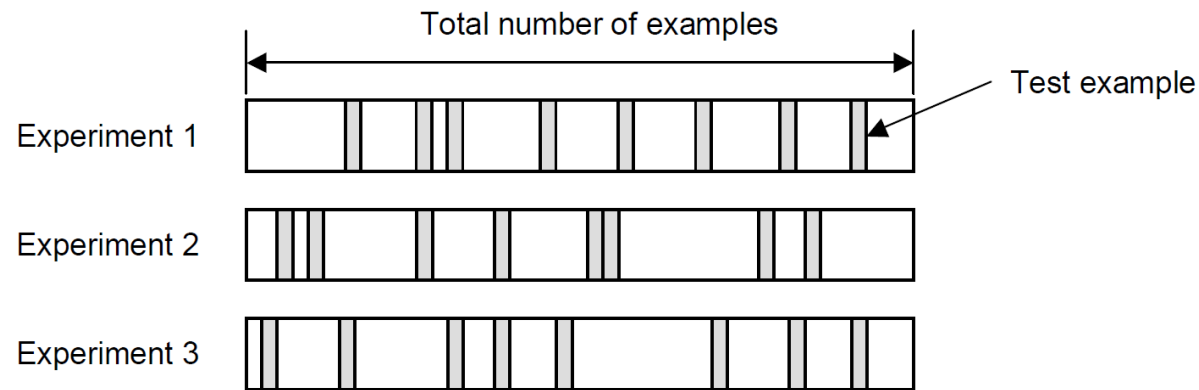


Holdout

- Drawbacks
 - For small training sets, setting aside a subset may be infeasible
 - For a single train-and-test experiment, the holdout estimate of error rate will be misleading if we happen to get an '*unfortunate*' split
- Alternatives: a family of resampling methods
 - Cross Validation
 - Random Subsampling
 - K-Fold Cross-Validation
 - Leave-one-out Cross-Validation

Random Subsampling

- **Random Subsampling performs K data splits of the dataset**
 - Random splits of (fixed) no. of examples without replacement
 - Each random train/test split: retrain classifier and estimate E_i with the test split

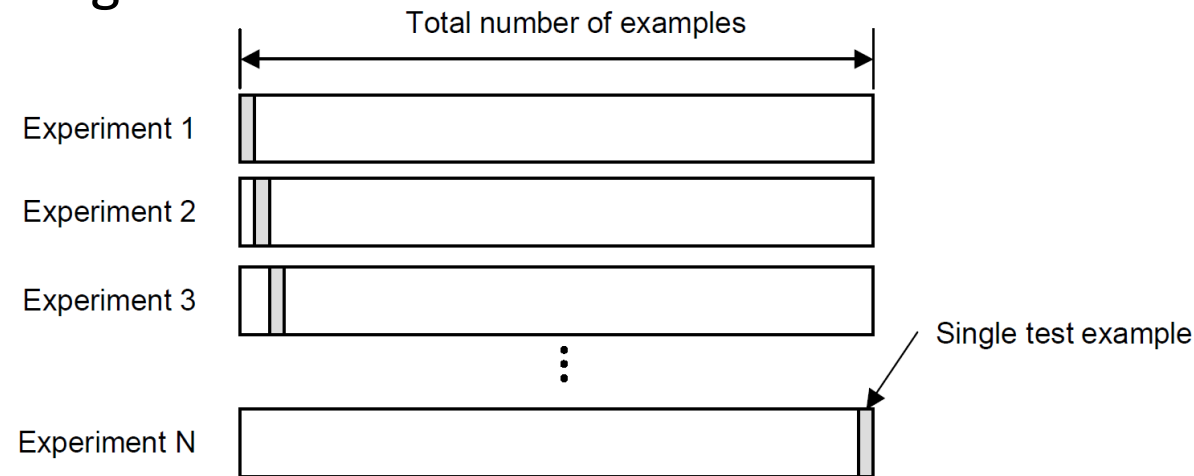


- **The true error estimate is obtained as the average of the separate estimates E_i**
 - This estimate is significantly better than the holdout estimate

$$E = \frac{1}{K} \sum_{i=1}^K E_i$$

Leave-one-out Cross Validation

- **Leave-one-out is the degenerate case of K-Fold Cross Validation, where K is chosen as the total number of examples**
 - For a dataset with N examples, perform N experiments
 - For each experiment use N-1 examples for training and the remaining example for testing

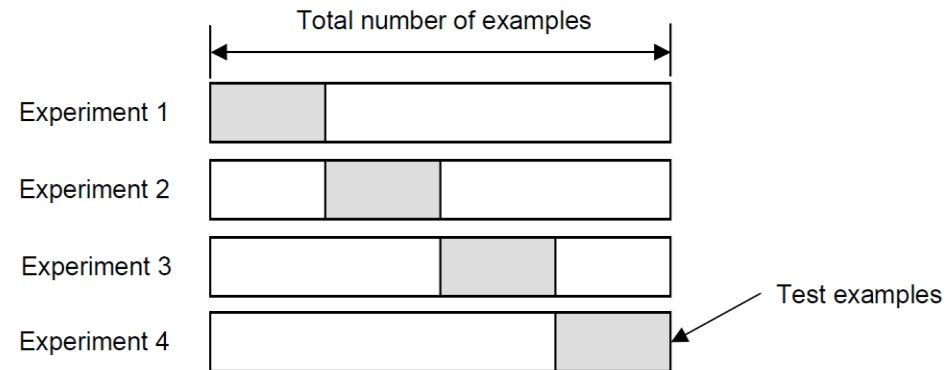


- **As usual, the true error is estimated as the average error rate on test examples**

$$E = \frac{1}{N} \sum_{i=1}^N E_i$$

Validation Method: K-Fold Cross-validation

- Randomly shuffle the dataset and create a K-fold partition
 - For each of K experiments, use K-1 folds for training and the remaining one for testing

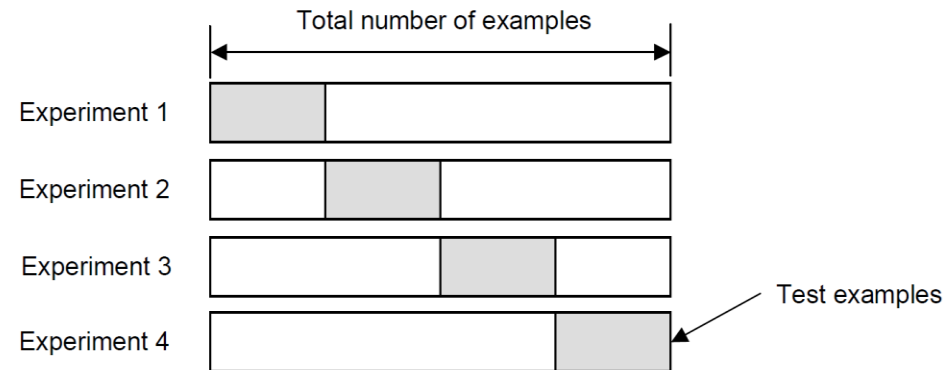


- True error is estimated as the average error rate over the validation subsets

$$E = \frac{1}{K} \sum_{i=1}^K E_i$$

Validation Method: K-Fold Cross-validation

- Create a K-fold partition of the dataset
 - For each of K experiments, use K-1 folds for training and the remaining one for testing



- True error is estimated as the average error rate

$$E = \frac{1}{K} \sum_{i=1}^K E_i$$

How many folds are needed?

- Large number of folds
 - + smaller bias of the true error rate estimator
 - larger variance of the true error rate estimator
 - higher computational time (many experiments)
- Small number of folds
 - + lower computation time
 - + smaller variance
 - larger bias
- In practice, the choice of the number of folds depends on the size of the dataset
 - For large datasets, even 3-Fold Cross Validation is reasonable
 - For very sparse datasets, '*leave-one-out*' is beneficial
- A common choice for K-Fold Cross Validation is $K=10$

Three-way data splits

- **If model selection and true error estimates are to be computed simultaneously, the data needs to be divided into three disjoint sets**
 - **Training set:** a set of examples used for learning: to fit the parameters of the classifier
 - In the MLP case, we would use the training set to find the “optimal” weights with the back-prop rule
 - **Validation (dev) set:** a set of examples used to tune the parameters of a classifier
 - In the MLP case, we would use the validation set to find the “optimal” number of hidden units or determine a stopping point for the back propagation algorithm
 - **Test set:** a set of examples used only to assess the performance of a fully-trained classifier
 - In the MLP case, we would use the test to estimate the error rate after we have chosen the final model (MLP size and actual weights)
 - After assessing the final model with the test set, YOU MUST NOT further tune the model

Three-way data splits

- **Why separate test and validation sets?**

- The error rate estimate of the final model on validation data will be biased (smaller than the true error rate) since the validation set is used to select the final model
- After assessing the final model with the test set, YOU MUST NOT tune the model any further

- **Procedure outline**

1. Divide the available data into training, validation and test set
2. Select architecture and training parameters
3. Train the model using the training set
4. Evaluate the model using the validation set
5. Repeat steps 2 through 4 using different architectures and training parameters
6. Select the best model and train it using data from the training and validation set
7. Assess this final model using the test set

This outline assumes a holdout method. Similar for Cross-Validation

Debugging ML Algorithms

- Motivating Example : Regularized Logistic regression [a.k.a. Bayesian Logistic Regression (BLR)]
 - Binary classification problem
 - Often encountered in computer vision: face/not face OR spam/not spam

$$\max_{\theta} \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}, \theta) - \lambda ||\theta||^2$$

Typical objective function
for optimization

- BLR with gradient descent generates a dev error of 20%
 - What to do next?

How to Debug an ML Algorithm?

Hit and Try and Pray to God!

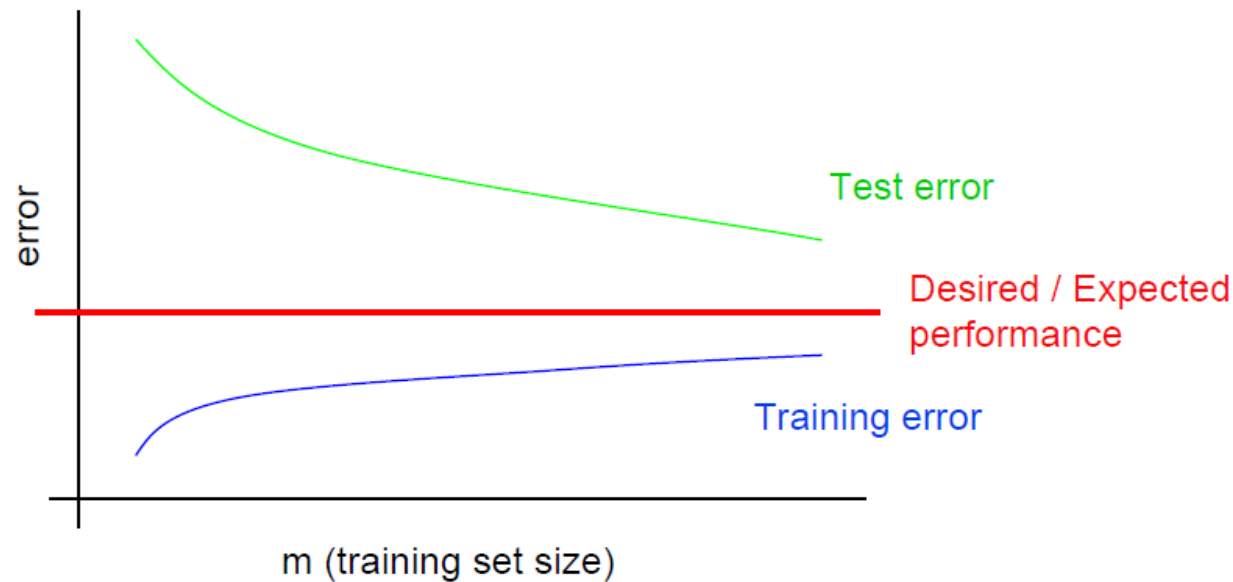
- Try getting more training examples.
- Try a smaller set of features.
- Try a larger set of features.
- Try changing the features.
- Run gradient descent for more iterations.
- Try Newton's method.
- Use a different value for λ .
- Try using an SVM.

Systematic Diagnosis

- Analyse variance/bias
- Analyse the optimization algorithm
- Analyse the objective function

Bias vs. Variance Analysis

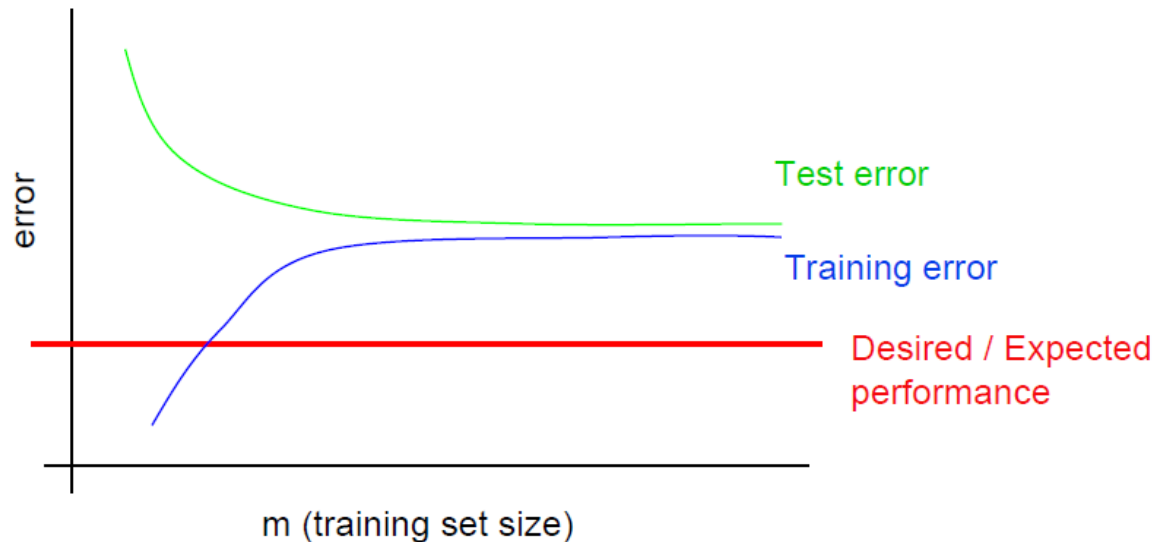
- Typical learning curve for high variance:



- Test error still decreasing as training set size increases.
 - Suggests a larger training set will help.
- Large gap between training and test error
 - Likely overfitting

Bias vs. Variance Analysis

- Typical learning curve for high bias:



Similar analyses could be done with optimization algorithm and objective function.

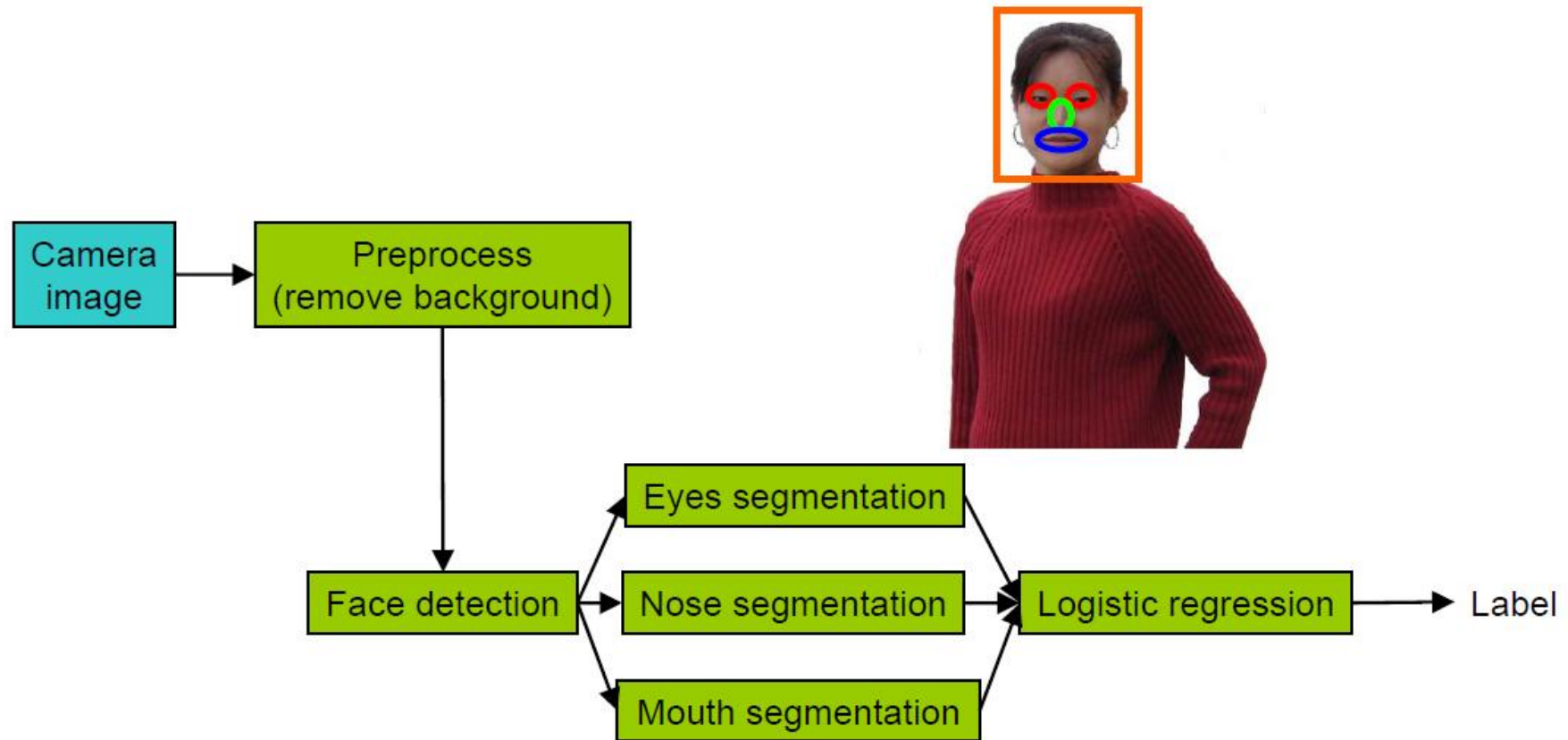
- Even training error is unacceptably high.
 - Features are not discriminative enough
- Small gap between training and test error.
 - Likely underfitting: a higher capacity model could be tried

Diagnostics for ML Algorithms

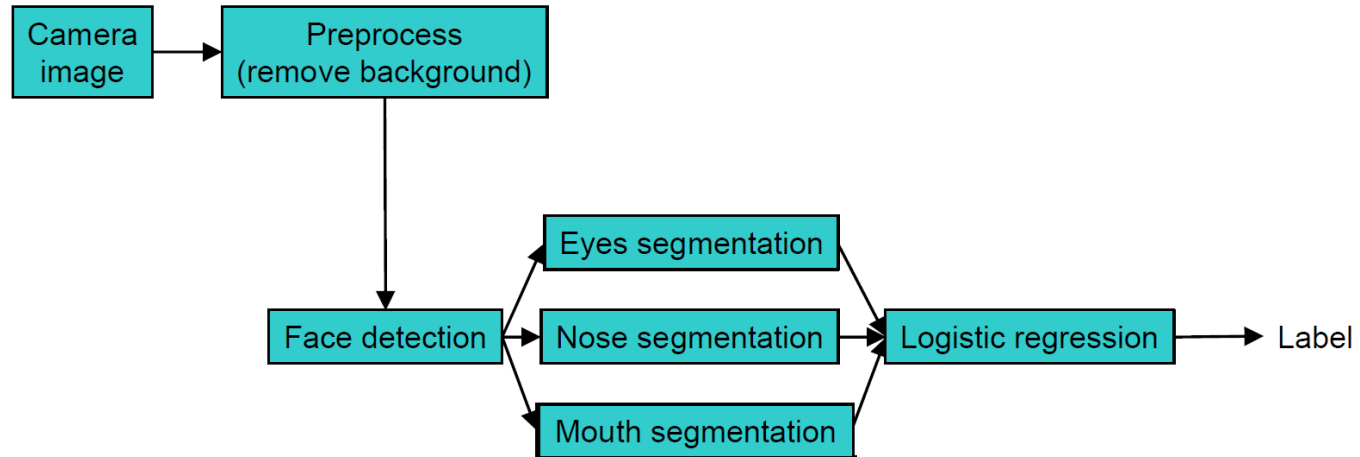
- Try getting more training examples.
 - Try a smaller set of features.
 - Try a larger set of features.
 - Try changing the features.
 - Run gradient descent for more iterations.
 - Try Newton's method instead of gradient descent.
 - Use a different value for reg. parameter λ .
 - Try using a different model (e.g., SVM).
- Fixes high variance.
 - Fixes high variance.
 - Fixes high bias.
 - Fixes high bias.
 - Fixes optimization algorithm.
 - Fixes optimization algorithm.
 - Fixes optimization objective.
 - Fixes optimization objective

Debugging ML Systems

- Many applications combine many different learning components into a “pipeline”, e.g., Face recognition from images [toy example].



Error Analysis



Ablative Analysis

- Error analysis tries to explain the difference between current performance and ideal performance.
- Ablative analysis tries to explain the difference between some baseline (much poorer) performance and current performance.

Suppose we threw in many features for training a Spam detector

- Question: How much did each of these features really help?

Ablative Analysis

[Toy Example: Spam Detection]

Simple logistic regression without any clever features get 94% performance.

Just what accounts for your improvement from 94 to 99.9%?

Ablative analysis: Remove components from your system one at a time, to see how it breaks.

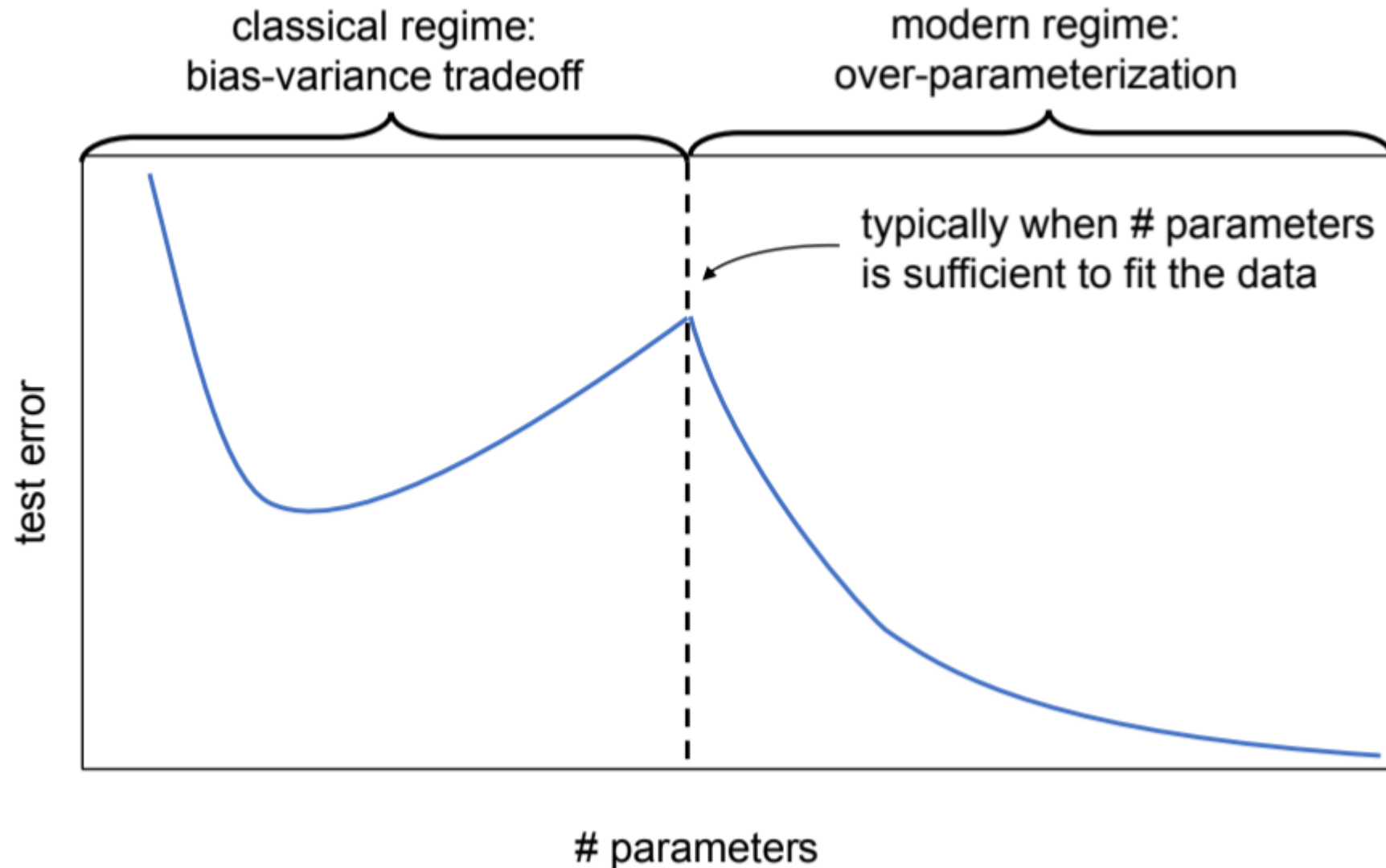
Component	Accuracy
Overall system	99.9%
Spelling correction	99.0
Sender host features	98.9%
Email header features	98.9%
Email text parser features	95%
Javascript parser	94.5%
Features from images	94.0%

[baseline]

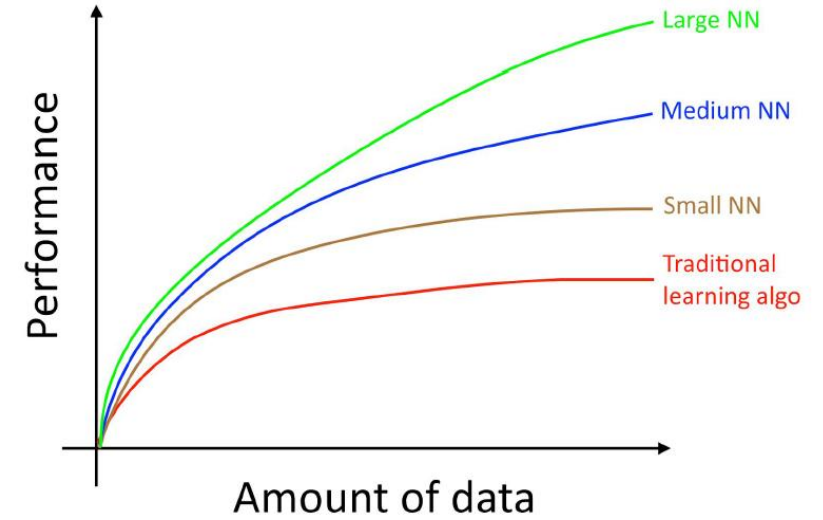
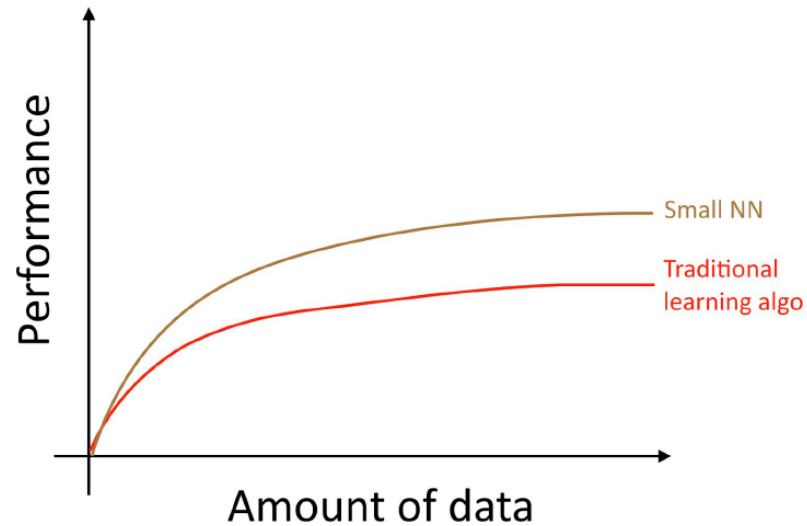
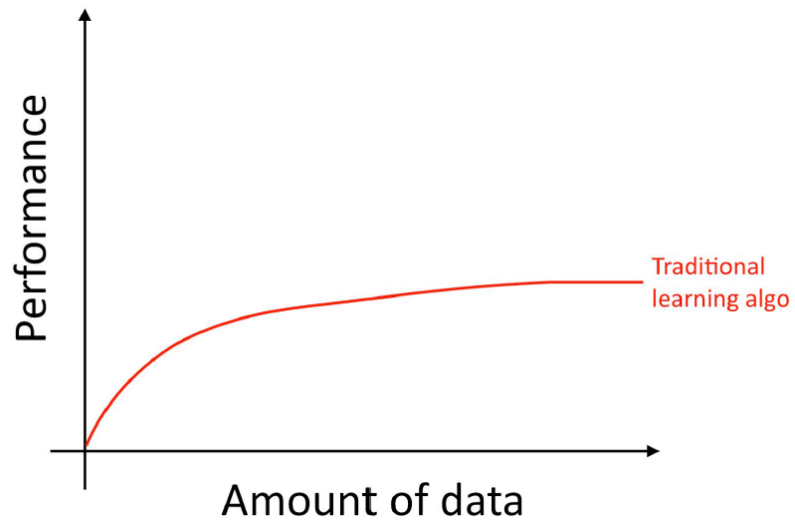
Conclusion: The email text parser features account for most of the improvement.

Recent Observations with the Bias-Variance Trade-off

The Double Descent



ML performance with Data



Setting Up your ML workspace

- **Training set** — Which you run your learning algorithm on.
- **Val / Dev (development) set** — Which you use to tune parameters, select features, and make other decisions regarding the learning algorithm. Sometimes also called the **hold-out cross validation set** .
- **Test set** — which you use to evaluate the performance of the algorithm, but not to make any decisions regarding what learning algorithm or parameters to use.

Try to pick test examples that reflect what you ultimately want to perform well on

Dev & Test set distributions

- If they come from the same distribution
 - Good performance on dev sets translates to good performance on test set
 - If not, the diagnosis is clear: we are overfitting the dev set
 - Solution: get more diverse dev set data
- If they come from different distributions, diagnosis of poor performance is harder
 - Overfitting dev set
 - Harder test set
 - Test set isn't harder, just different